Task B2 – Engineer an Adversarial Input (5 Marks, K4)

Run every program yourself and attach screenshots linked to the test they show. Numbers labelled **sandbox** were measured by me on a 2-core cloud machine while preparing this answer, so repeat them on your computer and report your own figures. `<fill>` marks data you must collect.

The SQL-injection test needs a **local test database created only for this assignment** (as the question says). I did not have MySQL in my sandbox, so I did not run `LoginDemo` against a database; the output below for it is what to expect. The query strings and the damaged-file test were actually run.

Test database (run once)

```
CREATE DATABASE testdb;  USE testdb;
CREATE TABLE users(id VARCHAR(20), pwd VARCHAR(40));
INSERT INTO users VALUES ('arun', 'secret');
```

Part 1 – Login: unsafe vs PreparedStatement (`LoginDemo.java`)

```
import java.sql.*;

/** Needs a LOCAL TEST database with: CREATE TABLE users(id
VARCHAR(20), pwd VARCHAR(40)); INSERT INTO users VALUES
('arun','secret'); */
public class LoginDemo {
    static boolean unsafe(Connection con, String id, String pwd)
throws SQLException {
        String sql = "SELECT * FROM users WHERE id='" + id + "' AND
pwd='" + pwd + "'";
        try (Statement st = con.createStatement(); ResultSet rs =
st.executeQuery(sql)) { return rs.next(); }
    }
    static boolean safe(Connection con, String id, String pwd) throws
SQLException {
        try (PreparedStatement ps = con.prepareStatement("SELECT 1
FROM users WHERE id = ? AND pwd = ?")) {
            ps.setString(1, id); ps.setString(2, pwd);
            try (ResultSet rs = ps.executeQuery()) { return rs.next();
}
        }
    }
    public static void main(String[] a) throws SQLException {
        try (Connection con =
DriverManager.getConnection("jdbc:mysql://localhost:3306/testdb",
"root", "password")) {
            String[][] inputs = {{"arun", "secret"}, {"arun",
"wrong"}, {"arun", "x' OR '1'='1"}, {"admin'#", "anything"}};
            for (String[] in : inputs)
                System.out.printf("id=%-10s pwd=%-16s unsafe=%-5b
```

```
safe=%b%n", in[0], in[1], unsafe(con, in[0], in[1]), safe(con, in[0],
in[1]));
        }
    }
}
```

Adversarial inputs and the SQL they produce (`Login.java`, sandbox):

Input (id, password)	SQL built by the unsafe version	Unsafe result	Safe (`PreparedStatement`)
`arun`, `secret`	`... id='arun' AND pwd='secret'`	accepted (valid)	accepted
`arun`, `wrong`	`... id='arun' AND pwd='wrong'`	rejected	rejected
`arun`, `x' OR '1'='1`	`... id='arun' AND pwd='x' OR '1'='1'`	**accepted without a valid password**	rejected
`admin'#`, `anything`	`... id='admin'#' AND pwd='anything'`	**accepted if user `admin` exists**; the password check is a comment	rejected

(The two "expected" results in the unsafe/safe columns are what the logic produces; confirm them with your database and screenshot the console output of `LoginDemo`. `<fill>`)

Part 2 – Damaged bus-pass file (`PassValidator.java`, sandbox output)

```
import java.util.ArrayList;
import java.util.List;

/** Checks one text line of the bus-pass file and explains what is
wrong. */
public class PassValidator {
    /** Returns null if the line is valid, otherwise the reason it is
damaged. */
    public static String check(String line) {
        if (line == null || line.isBlank()) return "empty line";
        String[] f = line.split(",", -1);
        if (f.length != 5) return "expected 5 fields but found " +
f.length;
        for (int i = 0; i < 3; i++) if (f[i].isBlank()) return "field
" + (i + 1) + " is empty";
        try { Integer.parseInt(f[3].trim()); } catch
(NumberFormatException e) { return "route is not an integer: '" + f[3]
+ "'"; }
```

```
        try {
            double amt = Double.parseDouble(f[4].trim());
            if (amt <= 0 || Double.isNaN(amt)) return "amount must be
positive: " + f[4];
        } catch (NumberFormatException e) { return "amount is not a
number: '" + f[4] + "'"; }
        return null;
    }
    public static void main(String[] a) {
        String[] lines = {
            "P1,192311001,Arun,12,4500.0",          // valid
            "P2,192311002,Divya,7",                 // wrong number of
fields
            "P3,192311003,Meena,12,abc",            // non-numeric
amount
            "",                                     // empty line
            "P5,192311005,Kumar,x,100",             // bad route
            "P6,192311006,Rani,3,-50"               // negative amount
        };
        for (String l : lines) {
            String r = check(l);
            System.out.println((r == null ? "OK       " : "REJECTED") +
" | '" + l + "'" + (r == null ? "" : "   -> " + r));
        }
    }
}
```

Line in the file	Result
`P1,192311001,Arun,12,4500.0`	OK
`P2,192311002,Divya,7`	REJECTED: expected 5 fields but found 4
`P3,192311003,Meena,12,abc`	REJECTED: amount is not a number: 'abc'
(empty line)	REJECTED: empty line
`P5,192311005,Kumar,x,100`	REJECTED: route is not an integer: 'x'
`P6,192311006,Rani,3,-50`	REJECTED: amount must be positive: -50

How the file store responds: `PassStore.find` skips any line without exactly 5 fields (wrong number of fields and the empty line), so the search carries on. It does **not** parse the amount, so `P3` with `abc` would be *returned* as a valid pass; a bad amount is only caught by the validator, which a robust program must call when reading or before adding.

What each adversarial input reveals about the original design

- `x' OR '1'='1` shows that **building SQL by concatenation lets data become code**; the root cause is mixing SQL and input, not a missing check on one field.

- `admin'#` shows that a **single quote or comment character in a field is enough** to remove part of the query, so a blacklist of a few words does not work.
- Wrong field count and empty line: the **file format has no checksum or length marker**, so damage can only be detected by structure; skipping silently hides the problem unless it is logged.
- `abc` as amount: **type checking was missing from the read path**; the file store accepts a line as long as the field count is right, so errors surface later (e.g. at `Double.parseDouble` in the loader).

Alternative concept and complexity

Escaping quotes or validating with a regular expression instead of `PreparedStatement` (`RegexValidation.java`, sandbox output):

```
import java.util.regex.Pattern;

/** Weaker alternatives to PreparedStatement. */
public class RegexValidation {
    static String escapeQuotes(String s) { return s.replace("'",
"''"); }      // escaping
    static final Pattern SAFE = Pattern.compile("[A-Za-z0-9_@.]
{1,30}");       // whitelist validation
    public static void main(String[] a) {
        String evil = "x' OR '1'='1";
        System.out.println("escaped    : " + escapeQuotes(evil));
        System.out.println("whitelist  : " +
SAFE.matcher(evil).matches() + " (rejected)");
        System.out.println("But a legitimate password like  P@ss'word
is also rejected by the whitelist: " + !
SAFE.matcher("P@ss'word").matches());
        // Backslash problem: with MySQL default settings, \\ is an
escape character, so escaping only quotes can fail:
        String tricky = "\\' OR 1=1 #";
        System.out.println("tricky     : " + escapeQuotes(tricky));
    }
}
```

```
escaped    : x'' OR ''1''=''1
whitelist  : false (rejected)
But a legitimate password like  P@ss'word  is also rejected by the
whitelist: true
tricky     : \'' OR 1=1 #
```

Why this is weaker:

- **Escaping** depends on the database's rules (MySQL treats the backslash as an escape character by default, so `\'` can undo your escaping, as the last line shows, and character sets and numeric fields create more holes). Every query must remember to escape every value, and a single missed place is an injection point.

- **Whitelist validation** rejects legitimate input (a password such as `P@ss'word`), and a whitelist loose enough for real names and passwords can still let dangerous input through.
- **`PreparedStatement`** separates code and data in the protocol itself, so no input value can change the query structure, whatever characters it contains.

Time cost per query: a regex check or escaping is **O(L)** for an input of length L (every character examined once), and a regex with nested quantifiers can even be exponential (ReDoS). A `PreparedStatement` sends the L characters of each parameter as data (also O(L) to send), but the SQL text is parsed once and the statement can be *reused* many times with different parameters, so for repeated queries it is as fast or faster. Therefore safety does not cost time: the safe version is the faster or equal one.